

HTTP, REST APIs, DTOs & Validation

Complete beginner-friendly notes. Learn what is happening, why it matters, then build a Book Catalog CRUD API.

TODAY'S GOAL

By the end of Day 2, you should be able to design a clean REST API, understand HTTP status codes, accept JSON input, validate it, and return DTO-based responses.

UNDERSTAND HTTP + REST	DESIGN Endpoints + responses	BUILD Book CRUD API
VALIDATE Reject bad input	TEST Success + error cases	DOCUMENT README + API behavior

Assumption: you know basic Java and completed Day 1.

Before learning REST, understand HTTP. HTTP is the set of rules used when one program talks to another over a network.

When you open a website or call an API, there is usually a simple conversation: **client sends a request** → **server processes it** → **server sends a response**.

```
CLIENT
|
| GET /api/books
| Accept: application/json
v
SERVER
|
| 200 OK
| Content-Type: application/json
| [ ...books... ]
v
CLIENT
```

Think of it like ordering

You ask for something, the server decides what to do, and it sends something back. The request and response have a predictable structure.

Part	Example	Meaning
Method	GET	What action do I want?
URL	/api/books/10	Which resource?
Headers	Content-Type	Extra information about the request
Body	{"title": "Java"}	Data sent to the server, usually for POST/PUT/PATCH

Method	Simple meaning	Book API example
GET	Read	GET /api/books
POST	Create	POST /api/books
PUT	Replace/update	PUT /api/books/10
PATCH	Partially update	PATCH /api/books/10
DELETE	Delete	DELETE /api/books/10

Easy CRUD mapping

Create → POST, Read → GET, Update → PUT/PATCH, Delete → DELETE.

The status code tells the client what happened.

Code	Meaning	Use it when...
200 OK	Success	A normal request succeeded.
201 Created	Created	A new book was created.
204 No Content	Success, no body	Delete succeeded and nothing needs returning.
400 Bad Request	Invalid request	Input is malformed or validation fails.
404 Not Found	Resource missing	Book ID does not exist.
409 Conflict	Conflict	Request conflicts with current data, such as duplicate ISBN.
500 Internal Server Error	Server problem	Unexpected failure on the server.

REST is a style for designing APIs. The important beginner idea is: **think in resources and use HTTP methods to act on those resources.**

A book is a resource. A collection of books is another useful way to think about that resource.

Less REST-like	Cleaner REST design
/getAllBooks	GET /api/books
/createBook	POST /api/books
/getBook?id=10	GET /api/books/10
/deleteBook?id=10	DELETE /api/books/10

Use nouns for resources. Usually use plural names for collections.

```
GET    /api/books
GET    /api/books/10
POST   /api/books
PUT    /api/books/10
DELETE /api/books/10
```

Avoid putting the action into the path when the HTTP method already expresses the action.

Good mental model

URL = **which thing?** HTTP method = **what do I want to do?**

Use	Example	Meaning
Path variable	/api/books/10	10 identifies the specific book.
Request parameter	/api/books?author=Joshua	Optional instructions for filtering/searching.

Annotation	Plain English
@RestController	This class handles API requests and returns response data.
@RequestMapping	Sets a common starting path for a controller.
@GetMapping	Handles GET requests.
@PostMapping	Handles POST requests.
@PutMapping	Handles PUT requests.
@PatchMapping	Handles PATCH requests.
@DeleteMapping	Handles DELETE requests.
@PathVariable	Reads a value from the URL path.
@RequestParam	Reads a value after ? in the URL.
@RequestBody	Reads JSON/data sent inside the request body.

```
@GetMapping("/api/books/{id}")
public BookResponse getBook(@PathVariable Long id) {
    // id comes from /api/books/10
}
```

If the client calls `/api/books/10`, Spring gives your method `id = 10`.

```
@GetMapping("/api/books")
public List<BookResponse> search(
    @RequestParam(required = false) String author) {
    // /api/books?author=Joshua
}
```

Request parameters are useful for optional filtering, sorting, searching, or pagination.

POST and PUT/PATCH commonly send data in the request body.

```
POST /api/books
Content-Type: application/json

{
  "title": "Effective Java",
  "author": "Joshua Bloch",
  "price": 650,
  "isbn": "9780134685991"
}
```

Spring can convert that JSON into a Java object for your controller.

```
@PostMapping("/api/books")
public BookResponse create(@RequestBody BookCreateRequest request) {
    return service.create(request);
}
```

DTO means **Data Transfer Object**. For Day 2, think of a DTO as a Java class used to carry data into or out of your API.

A common mistake is returning your database/entity object directly. Keeping API data separate gives you control over what clients can send and what they can see.

DTO	Purpose
BookCreateRequest	Data allowed when creating a book.
BookUpdateRequest	Data allowed when updating a book.
BookResponse	Data your API sends back to the client.

Simple rule

Request DTO = what the client is allowed to send. Response DTO = what your API chooses to return.

```
public class BookCreateRequest {
    private String title;
    private String author;
    private BigDecimal price;
    private String isbn;

    // constructor + getters/setters
}
```

```
public class BookUpdateRequest {
    private String title;
    private String author;
    private BigDecimal price;
    private String isbn;

    // constructor + getters/setters
}
```

```
public class BookResponse {
    private Long id;
    private String title;
    private String author;
    private BigDecimal price;
    private String isbn;

    // constructor + getters/setters
}
```

For learning, ordinary Java classes are enough. Later you can explore records, Lombok, mapping libraries, and more advanced DTO patterns.

Operation	Endpoint	Typical response
List books	GET /api/books	200 + list/page
Get one	GET /api/books/{id}	200 + book or 404
Create	POST /api/books	201 + created book
Update	PUT /api/books/{id}	200 + updated book or 404
Delete	DELETE /api/books/{id}	204 or 404

Never assume the client sends perfect data. Validation checks incoming data before your application tries to use it.

Example: a book with an empty title or a negative price should not be accepted.

Think of validation as a gate

Bad input is stopped at the entrance instead of being allowed deeper into your application.

Spring Boot can use Jakarta Validation annotations to describe rules directly on DTO fields.

Annotation	Simple meaning	Example
@NotBlank	Text must contain something.	@NotBlank String title
@NotNull	Value cannot be null.	@NotNull BigDecimal price
@Size	Text/list size must fit a range.	@Size(max=100)
@Positive	Number must be greater than 0.	@Positive BigDecimal price
@Min / @Max	Number must stay within limits.	@Min(1)
@Pattern	Text must match a pattern.	Useful for structured strings

```
public class BookCreateRequest {  
  
    @NotBlank  
    private String title;  
  
    @NotBlank  
    private String author;  
  
    @NotNull  
    @Positive  
    private BigDecimal price;  
  
    @NotBlank  
    private String isbn;  
}
```

Then tell Spring to validate the incoming request with `@Valid`:

```
@PostMapping("/api/books")  
public ResponseEntity<BookResponse> create(  
    @Valid @RequestBody BookCreateRequest request) {  
  
    return ResponseEntity  
        .status(HttpStatus.CREATED)  
        .body(service.create(request));  
}
```

What happens?

If validation passes, your method runs. If validation fails, Spring can reject the request before your business logic runs.

POST /api/books

```
{  
  "title": "",  
  "author": "Someone",  
  "price": -50,  
  "isbn": ""  
}
```

→ 400 Bad Request

Imagine your database has 100,000 books. Returning all of them in one response is inefficient. Pagination means returning a smaller portion at a time.

```
GET /api/books?page=0&size=20
```

Here, **page=0** means the first page and **size=20** means up to 20 books.

Spring Data later provides powerful pagination support through `Pageable`. For Day 2, understand the idea first.

```
GET /api/books?sort=price,asc  
GET /api/books?sort=title,desc
```

Sorting tells the server how to order the results.

```
GET /api/books?author=Joshua  
GET /api/books?minPrice=100&maxPrice=500
```

Filtering asks for only the resources matching certain conditions.

Design rule

Pagination, sorting, and filtering are usually request parameters because they modify how a collection is returned; they do not identify a different resource.

An operation is **idempotent** when repeating the same request has the same intended final effect as doing it once.

Method	Usually idempotent?	Simple idea
GET	Yes	Reading does not change the resource.
PUT	Yes	Setting the resource to the same state repeatedly is intended to produce the same final state.
DELETE	Yes	After the resource is gone, repeating the delete does not normally change the final state.
POST	Usually no	Repeating create may create multiple resources.

Important nuance

Idempotent does not mean the response must be identical every time. It is about the intended effect on the resource.

A REST-style server should not depend on remembering the previous request in order to understand the next request. Each request should contain the information needed to process it.

Simple example

A request should not mean “continue what I asked you to do five requests ago.” It should be understandable on its own.

24 404 vs 400 vs 409 — know the difference

Situation	Status	Example
Request is invalid	400	Price is negative or required title is missing.
Book does not exist	404	GET /api/books/9999 when ID 9999 is absent.
Request conflicts with existing state	409	Trying to create a book with an ISBN that must be unique.

25 Consistent error responses

Avoid returning a completely different error shape from every endpoint. A consistent format is easier for frontend developers and API consumers.

```
{
  "timestamp": "2026-09-08T10:30:00Z",
  "status": 400,
  "message": "Validation failed",
  "errors": {
    "title": "Title must not be blank",
    "price": "Price must be positive"
  }
}
```

The exact structure can vary. The important idea is consistency.

26 PUT vs PATCH

PUT	Replace/update the resource representation. Think: “Here is the new version.”
PATCH	Change only selected fields. Think: “Change these parts.”

```
src/main/java/com/example/bookapi/  
■■■ controller/  
■   ■■■ BookController.java  
■■■ service/  
■   ■■■ BookService.java  
■■■ dto/  
■   ■■■ BookCreateRequest.java  
■   ■■■ BookUpdateRequest.java  
■   ■■■ BookResponse.java  
■■■ BookApiApplication.java
```

For Day 2, you can keep the data in memory instead of adding a database if you want to focus on HTTP, DTOs, and validation.

```
@RestController  
@RequestMapping("/api/books")  
public class BookController {  
  
    private final BookService service;  
  
    public BookController(BookService service) {  
        this.service = service;  
    }  
  
    @GetMapping  
    public List<BookResponse> getAll() {  
        return service.getAll();  
    }  
  
    @GetMapping("/{id}")  
    public BookResponse getOne(@PathVariable Long id) {  
        return service.getOne(id);  
    }  
  
    @PostMapping  
    public ResponseEntity<BookResponse> create(  
        @Valid @RequestBody BookCreateRequest request) {  
        BookResponse created = service.create(request);  
        return ResponseEntity.status(HttpStatus.CREATED).body(created);  
    }  
}
```

```
@PutMapping("/{id}")
public BookResponse update(
    @PathVariable Long id,
    @Valid @RequestBody BookUpdateRequest request) {
    return service.update(id, request);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
}
```

The Controller should not contain all the application logic. It should receive input, call the Service, and return the result.

```
@Service
public class BookService {

    public List<BookResponse> getAll() {
        // return books
    }

    public BookResponse getOne(Long id) {
        // find book or throw not-found error
    }

    public BookResponse create(BookCreateRequest request) {
        // validate business rules
        // create book
        // return response DTO
    }

    public BookResponse update(Long id, BookUpdateRequest request) {
        // find, update, return
    }

    public void delete(Long id) {
        // find, delete
    }
}
```

Separation of responsibility

Controller = HTTP. Service = application work. DTO = data shape. This separation is one of the most important habits to build early.

For every endpoint, test at least three kinds of cases: **success**, **invalid input**, and **missing/conflicting data**.

Test	Expected
GET all books	200 OK
GET existing book	200 OK
GET missing book	404 Not Found
POST valid book	201 Created
POST blank title	400 Bad Request
POST negative price	400 Bad Request
PUT existing book	200 OK
DELETE existing book	204 No Content
DELETE missing book	404 Not Found

```
POST http://localhost:8080/api/books
Content-Type: application/json

{
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "price": 499,
  "isbn": "9780132350884"
}
```

```
POST http://localhost:8080/api/books
Content-Type: application/json

{
  "title": "",
  "author": "",
  "price": -10,
  "isbn": ""
}

Expected: 400 Bad Request
```

```
# Book Catalog API

## Base URL
http://localhost:8080

## Endpoints

GET    /api/books
GET    /api/books/{id}
POST   /api/books
PUT    /api/books/{id}
DELETE /api/books/{id}

## Create a book
POST /api/books
Content-Type: application/json

{
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "price": 499,
  "isbn": "9780132350884"
}
```

Also document validation rules, status codes, pagination parameters, and how to run the project.

#	Question
1	What is HTTP?
2	What is the difference between GET and POST?
3	PUT vs PATCH?
4	What does 201 Created mean?
5	400 vs 404 vs 409?
6	What is REST?
7	Why use nouns in REST URLs?
8	What is a path variable?
9	What is a request parameter?
10	What does @RequestBody do?
11	Why use DTOs?
12	What is @Valid?
13	Why validate incoming data?
14	What is pagination?

15	What does idempotent mean?
16	What does stateless mean?
17	Why separate Controller and Service?
18	What should happen when a book ID does not exist?

Build the complete **Book Catalog CRUD API**. Work in this order:

Step	Build
1	Create project + packages.
2	Create BookCreateRequest, BookUpdateRequest, BookResponse.
3	Create Controller and Service.
4	Implement GET all / GET one.
5	Implement POST + 201 Created.
6	Implement PUT + DELETE.
7	Add validation.
8	Handle 404 and validation errors.
9	Add pagination/sorting/basic filtering.
10	Test all success and failure cases.
11	Update README and commit to Git.

- I can explain HTTP request and response without notes.
- I know GET, POST, PUT, PATCH and DELETE.
- I can choose sensible HTTP status codes.
- I can design REST URLs using resources.
- I understand @RequestBody, @PathVariable and @RequestParam.
- I can explain why DTOs are useful.
- I can add @Valid and validation annotations.
- I understand 400 vs 404 vs 409.
- I understand pagination, filtering and sorting.
- I understand idempotency and statelessness.
- I can build the Book CRUD API from scratch.
- I tested invalid input instead of only successful requests.
- I documented the API and committed my work.

Day 2 success condition

Close this PDF and build the Book Catalog API yourself. If you can explain why each endpoint uses its method, status code, DTO, and validation rule, you have completed the day successfully.